مستند عن تكليف مهندس برمجيات أول (واجهات خلفية)

في هذا المستند سوف ابدا بتحليل المتطلبات الوظيفة وهي مبنية على افتراضات وبعد ذلك سوف اتحدث عن المتطلبات غير الوظيفة وطريقة الحل وتجد أيضا تصميم النظام وقاعدة بيانات CMS وبعدها طريقة تشغيل النظام وفي النهاية سوف أتكلم عن التحديات والتحسينات المقترحة.

تحليل المتطلبات الوظيفيةFunctional:

1 - نظام CMS يدير محتوى البرامج:

a. يسمح فقط لموظفين اداره المحتوى والمحررين.
b. يمكنهم انشاء، تعديل، حذف برامج وتصنيفها مثلا بودكاست أو أفلام وثائقية وتكون بإحدى الحالات التأليه، منشوره "published" او نسخه اوليه "draft" وتصنيفها مثلا حوارات، أعمال، وثائقية، الى اخره.
c. بعد انشاء البرنامج يمكن للمستخدم انشاء، تعديل، حذف حلقات مرتبطة بالبرامج.
d. يمكن للمستخدم رفع وحذف ملفات المحتوى (Media Files) مثل الفيديوهات والصوت وربطها بالحلقات.
e. استيراد المحتوى من انظمه أخرى مثل آبل بودكاست وتخزين بياناته الوصفية في النظام بطريقه تتماثل مع نموذج بيانات النظام (Data Modeling).

2 - نظام Discovery نظام البحث عن المحتوى:

a. يسمح لعموم المستخدمين.
b. عرض فقط البرامج المنشورة "published".
c. عرض تصنيفات البرامج وكم برنامج في كل تصنيف مثلا حوارات برنامجين، أعمال برنامج واحد وهكذا.
d. عرض جميع البرامج مع قابليه تخصيص البحث مثلا عرض برامج الحوارات فقط او بناء على نوع البرنامج مثلا برامج البودكاست فقط وقابله للترتيب على حسب متى انشأت او عدد الحلقات او على العنوان.

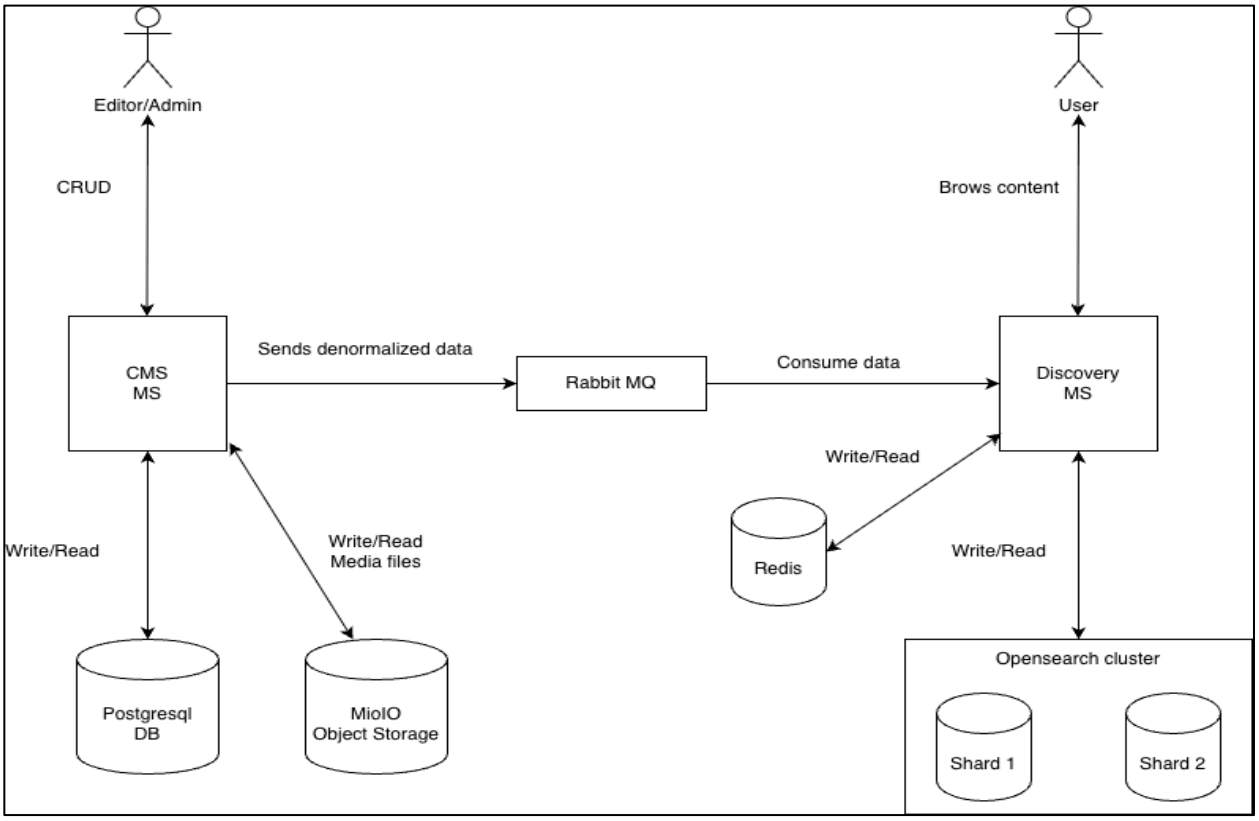
e. عرض تفاصيل البرنامج مع حلقاته مع قابله تخصيص البحث مثلا ترتيب الحلقات تصاعدين او تنازلين بحث عن حلقات الموسم الثاني فقط وهكذا،
f. بحث حر Free text searchيمكن للمستخدم البحث بكلمات مفتحيه اما ان تكون اسم البرنامج او الحلقة او تصنيف البرامج مثلا "فنجان" او "أعمال" او " كيف نجعل القرآن أسلوب حياة ".

تحليل المتطلبات غير الوظيفية Non Functional ومعاير الحل والتصميم:

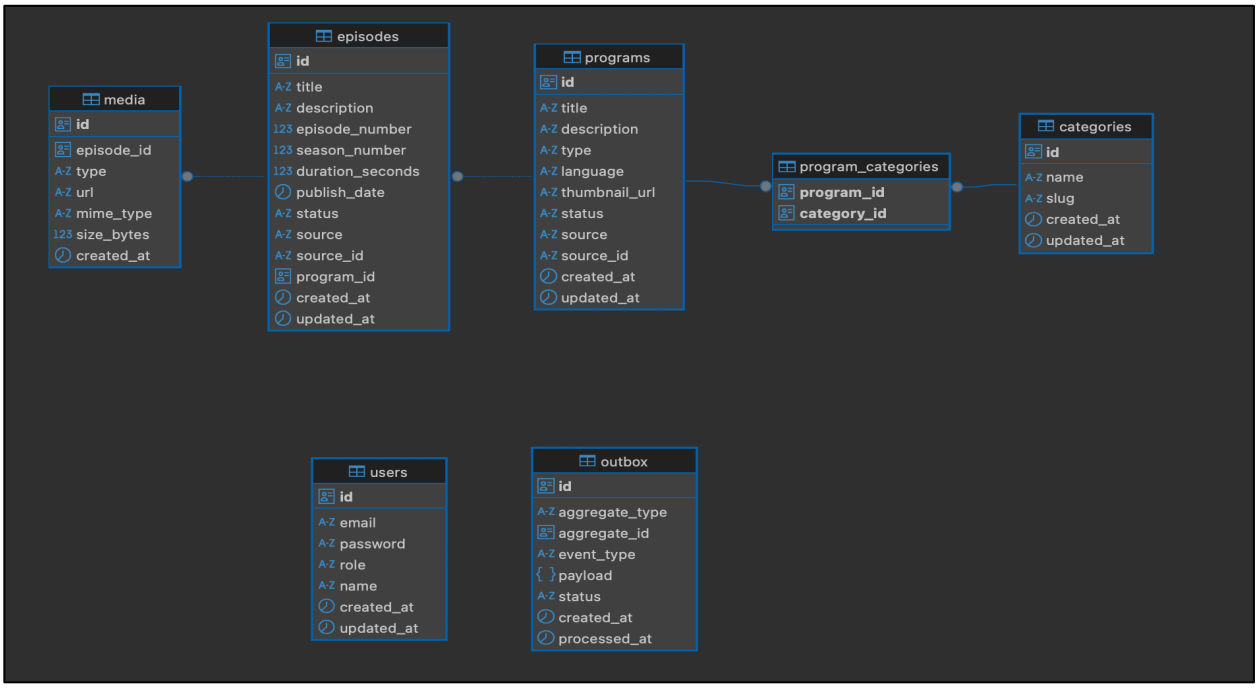
- عدد مستخدمين النظام هو 10 مليون في الساعة أي حوالي 2.8kفي الثانية سوف افترض ان معظمها أي أكثر من 90% قادم من البحث عن المحتوى أي من Discovery service.
- تم تصميم وتنفيذ النظام على ان يكون Microservices بسبب ان عدد مستخدمين النظام غير متكافئ بين CMS وDiscovery وسوف يكون التوسع في نظام discovery أكثر بكثير من CMS وتم استعمال NestJS للنظامين.
- الربط بين النظامين سوف يكون باستخدام أنظمه الرسائل MQ مما يقلل الترابط بين النظامين Low coupling وأكثر Resilient وذلك بتباع Outbox pattern.
- بسبب ان النظامين مربوطة باستخدام MQ فان قاعدة بيانات discovery service لن تكون متزامنة مع نظام CMS في نفسه اللحظة بمعنى انها Eventual consistent وهذا مقبول لأنها عرض بيانات فقط.
- تم استخدام OpenSearch كقاعدة بيانات اساسيه في Discovery Service وذلك يسمح لنا بالتوسع الافقي Horizontal Scalingو البحث الحر Free text search.
- تم استخدام caching في Discovery service لتقليل الضغط على OpenSearch وتخزين الاستعلامات المتكررة مما يساعد النظام على تحمل مستخدمين أكثر. وعمليه حذف الcache باستخدام الطرقتين TTL وبالاعتماد على الرسال القادمة منCMS التي لها علاقة بالمحتوى المحفوظ في الذاكرة.
- في هذا تصميم إذا كان احدى الأنظمة لا يعمل فهو لا يوثر على مستخدمين النظام الاخر fault isolation.
- للتسهيل تم استخدام انظمه الأمان فقط على CMS APIs.

- تم كتابة كود استيراد المحتوى بنمط Strategy pattern وذلك يسمح لنا مستقبلا بإضافة مصادر أخرى ومن غير ان نؤثر على المصادر الحالية Open/Close principal وتم تنفيذ الاستيراد من apple podcast.
- تم توثيق Endpoints باستخدام swagger.
- بالتصميم ادناه يمكن للنظام ان يتحمل هذا الكم من المستخدمين.

التصميم المعتمد والمنفذ:



تصميم علاقات قاعده بيانات CMS، ER Diagram:



طريقة تشغيل النظام:

بعد فك الضغط من الملف المرسل سوف تجد داخل applications folder مجلدين وملف CMS service وDiscovery service وdocker-compose

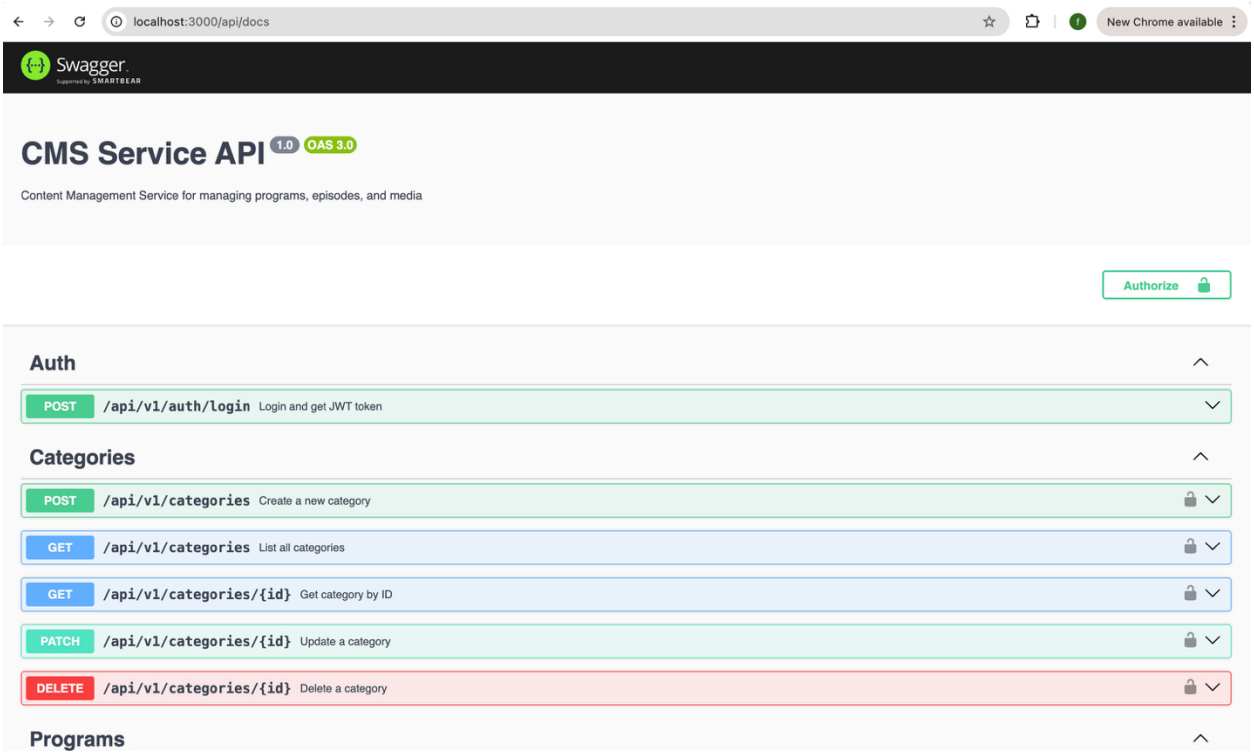
```
applications — -zsh — 91×24
faisal@Faisals-MacBook-Air applications % ls -l
total 8
drwxr-xr-x  12 faisal  staff   384 Mar  9 09:44 cms-service
drwxr-xr-x  11 faisal  staff   352 Mar  9 09:14 discovery-service
-rw-------@  1 faisal  staff  2089 Mar  9 06:08 docker-compose.yml
faisal@Faisals-MacBook-Air applications %
```

لتشغيل النظام اكتب هذا الامر ادناه:

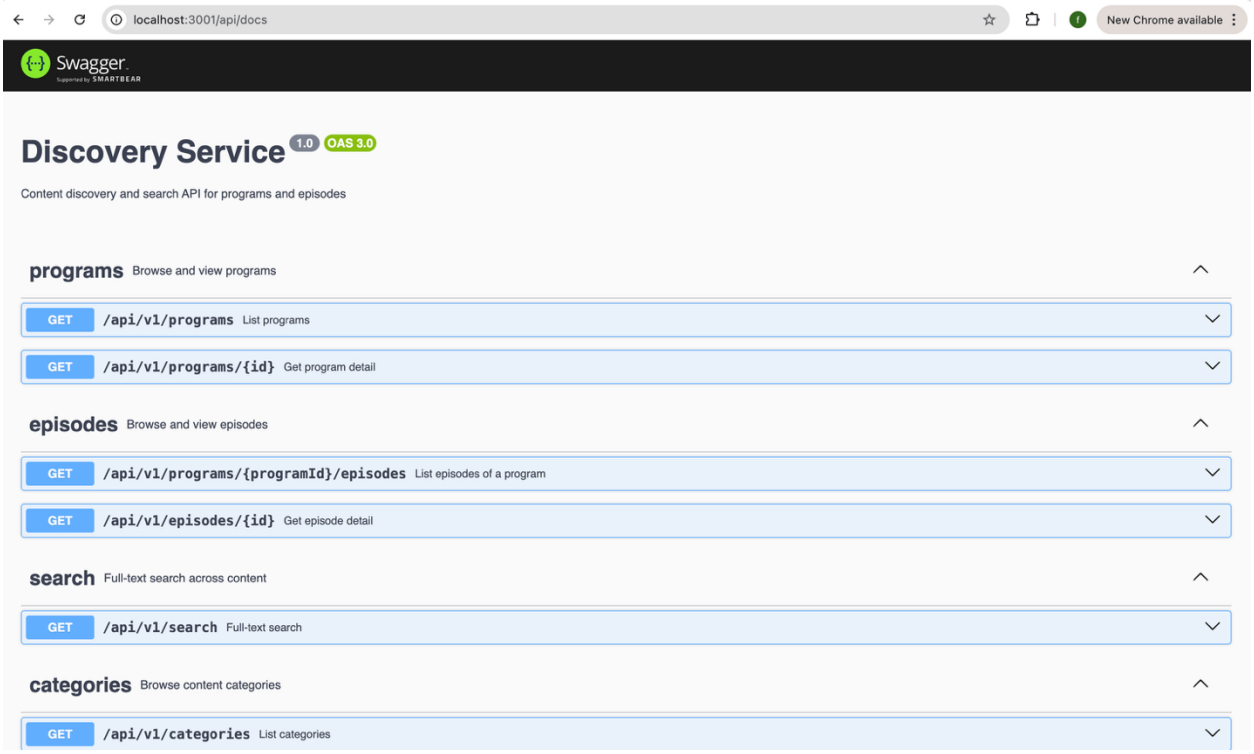
docker-compose up –build

بعد ذلك يمكنك استخدام swagger للنظامين وتجربة النظام:

نظام CMS هو: http://localhost:3000/api/docs



نظام Discovery هو:http://localhost:3001/api/docs



التحديات والتحسينات المقترحة:

من التحديات:

- متطلبات هذا التكليف عامه، اضطررت لبناء متطلبات مبنيه على افتراضات بحيث ما تخرج عن سياق فكره النظام الأساسية.
- تصميم نظام يتحمل 10m مستخدم في الساعة ليس بسيط يتطلب استخدام تقنيات كثيرة نسبيا.

من التحسينات المقترحة:

- بدل من جعل outbox table معتمد على background job يشتغل كل 5 ثواني بإمكان استخدام CDC وهذا يقول بتسريع بنقل البيانات من outbox table الى MQ من 5 ثواني الى أجزاء من الثانية لأنه يعتمد على Database logs ف أي تغير يحدث يتم التقاطه بشكل لحظي.
- لابد من ان نضع حد لعدد العمليات الموجة لنظام Discovery بالخصوص وذلك لحماية النظام ولتفادي أي استخدام خاطئ سوى كان ذلك متعمد DDOS او غير متعمد.